

Contexto do Projeto e Diretrizes para Estudo — Disponibilidade de Agenda

1. Contexto do projeto

Estou desenvolvendo uma API de agendamento com escopo inicialmente fechado, que posteriormente poderá ser utilizada como base para soluções destinadas a diferentes comércios e prestadores de serviços com regras de negócio semelhantes.

Um amigo desenvolverá um bot de atendimento utilizando n8n, que consumirá minha API. Por isso, quero manter uma separação clara de responsabilidades: a API deve conhecer apenas as regras e os dados pertencentes ao seu próprio domínio, sem assumir como o consumidor externo irá utilizar ou interpretar as informações.

A API poderá ser consumida, por exemplo, por um frontend, por um fluxo de automação ou por um agente de atendimento. Portanto, as respostas devem representar informações e regras do domínio de forma reutilizável, sem serem moldadas especificamente para um desses consumidores.

O projeto já está em desenvolvimento e as principais classes já existem. Não quero refatorar toda a estrutura sem necessidade. As alterações devem ser feitas apenas quando houver uma justificativa técnica ou de domínio clara.

2. Problema principal

Uma das funcionalidades do sistema será consultar a agenda de um prestador de serviço e retornar os intervalos de tempo em que um novo agendamento pode ser realizado, considerando:

- o expediente do prestador;
- os períodos já ocupados;
- a duração desejada para o novo agendamento;
- o período de dias solicitado para a pesquisa.

Não quero retornar slots discretos de agendamento. O objetivo é retornar **intervalos contínuos livres**.

Exemplo:

Expediente: 08:00 → 16:00

Compromissos: 11:00 → 11:40 12:00 → 13:30

Duração solicitada: 30 minutos

Resultado:

08:00 → 11:00

13:30 → 16:00

O intervalo é retornado integralmente quando possui duração igual ou superior à duração solicitada. Não quero dividi-lo em slots de 30 minutos.

3. Abstração de períodos

Estou utilizando uma abstração semelhante a:

```
record TimePeriod(DateTimeOffset Start, DateTimeOffset End);
```

Um `TimePeriod` representa um intervalo temporal.

No domínio, um intervalo válido deve obrigatoriamente possuir:

```
Start < End
```

Portanto, não quero permitir:

```
Start == End
```

nem:

```
Start > End
```

Os períodos ocupados são tratados como uma abstração de "ocupação", e não necessariamente como "compromissos". Isso permite que uma ocupação represente, por exemplo, um atendimento, uma folga, um bloqueio ou outro período durante o qual o prestador não pode receber um novo agendamento.

4. Função de listagem de horários disponíveis

Entradas

A funcionalidade recebe conceitualmente:

- identificador do prestador;
- duração desejada para o agendamento;
- período de busca, representado por um intervalo de dias;
- períodos ocupados da agenda.

O período de busca é tratado como uma sequência de dias, e não como um intervalo arbitrário de data e hora.

5. Regras da funcionalidade

Duração

1. A duração do agendamento deve ser expressa em minutos.
2. A duração mínima permitida é de 10 minutos.
3. A duração máxima permitida é de 90 minutos.
4. A duração deve ser múltipla de 5.

Portanto, valores válidos estão dentro do intervalo:

10, 15, 20, 25, ..., 90

Período da pesquisa

1. O período de busca deve possuir no mínimo 1 dia e no máximo 35 dias.
2. A pesquisa deve ser realizada por dias.
3. O início do período de busca não pode preceder o dia vigente no momento da requisição.

A regra acima significa apenas que a pesquisa não pode começar em uma data passada. Não é necessário, neste momento, eliminar horários que já tenham passado dentro do próprio dia atual.

Períodos ocupados

1. Os períodos ocupados devem possuir data e hora de início e fim.
2. Os períodos utilizados pelo algoritmo são considerados válidos e não sobrepostos.
3. Caso seja necessário garantir essa condição, a responsabilidade poderá ficar na implementação que fornece os períodos ocupados, por exemplo, no serviço responsável por implementar `GetBusyPeriodsAsync`.

A abstração que atualmente imagino para essa consulta é semelhante a:

```
GetBusyPeriodsAsync(  
    string calendarId,  
    DateTimeOffset start,  
    DateTimeOffset end  
);
```

Ela deve retornar uma coleção de `TimePeriod`.

Expediente

O prestador possui horários de início e fim de expediente, atualmente representados por `TimeOnly`.

Exemplo:

08:00 → 18:00

Também existem expedientes que atravessam a meia-noite:

23:30 → 06:00

Nesse caso, o expediente representa um único intervalo contínuo:

23:30 do dia D → 06:00 do dia D+1

A meia-noite não deve ser tratada como uma interrupção artificial do intervalo.

Da mesma forma, um compromisso que começa após a meia-noite pode pertencer temporalmente ao expediente iniciado no dia anterior.

6. Lógica pretendida

A lógica que estou desenvolvendo atualmente é a seguinte:

1. Receber o identificador do prestador, período de busca e duração desejada.
2. Consultar o prestador no repositório.
3. Obter seus dados relevantes para o cálculo, incluindo horário de início e fim do expediente e identificador do calendário.
4. Consultar os períodos ocupados no intervalo necessário por meio da abstração de calendário.
5. Percorrer o período de busca dia a dia.
6. Para cada dia, construir o intervalo temporal correspondente ao expediente daquele dia.
7. Se o expediente atravessar a meia-noite, seu fim deverá pertencer ao dia seguinte, preservando o expediente como um único intervalo contínuo.
8. Identificar os períodos ocupados que interferem nesse intervalo de trabalho.
9. Ordenar os períodos ocupados cronologicamente, caso essa garantia não faça parte do contrato da abstração que os fornece.
10. Percorrer os períodos ocupados e identificar os intervalos livres entre:
 - início do expediente e primeira ocupação;
 - fim de uma ocupação e início da próxima;
 - fim da última ocupação e fim do expediente.
11. Um intervalo livre somente deve ser retornado quando sua duração for maior ou igual à duração solicitada para o novo agendamento.
12. Os resultados devem ser agrupados por dia no retorno.

A ideia central é tratar o problema como uma operação sobre intervalos:

```
Período de trabalho
-
Períodos ocupados
=
Intervalos livres
```

seguido da filtragem:

```
duração do intervalo livre >= duração solicitada
```

O cálculo deve trabalhar com intervalos temporais contínuos. O agrupamento por dia pertence à estrutura da resposta e não deve obrigar o algoritmo interno a interromper um intervalo na meia-noite.

7. Formato esperado da saída

A resposta deve ser agrupada por data, contendo os intervalos disponíveis daquela data.

Exemplo conceitual:

```
{
  "Days": [
    {
      "Date": "2026-09-02",
      "TimePeriods": [
        {
          "Start": "08:00",
          "End": "11:00"
        },
        {
          "Start": "13:30",
          "End": "16:00"
        }
      ]
    }
  ]
}
```

Não quero retornar slots de duração fixa.

Um intervalo:

08:00 → 11:00

deve permanecer como um único intervalo, desde que sua duração seja suficiente para o serviço solicitado.

8. Decisões arquiteturais já tomadas

Quero que estas decisões sejam consideradas antes de sugerir mudanças:

- O projeto possui escopo de MVP fechado.
- Não quero adicionar regras de negócio futuras apenas por antecipação.
- Não quero criar uma solução excessivamente genérica.
- O objetivo é criar uma base simples que possa posteriormente receber novas regras de negócio.
- A API deve ser independente do contexto externo que a consome.
- Não quero que o domínio seja moldado especificamente para o frontend ou para o bot do n8n.
- Não quero retornar slots neste momento.
- O cálculo deve trabalhar com intervalos contínuos.
- Não quero permitir `Start == End`.
- Estou assumindo, por enquanto, que os períodos ocupados fornecidos ao algoritmo não possuem sobreposição.
- As respostas externas do sistema trabalharão com UTC.
- A API recebe informações temporais do calendário em UTC e precisa considerar corretamente o fuso associado aos dados de entrada.
- Pretendo utilizar `DateTimeOffset` para representar os instantes temporais relevantes.

9. Como quero estudar e desenvolver esta funcionalidade

Estou usando este projeto como oportunidade para desenvolver autonomia técnica.

Não quero simplesmente receber uma implementação pronta.

Quero primeiro entender:

1. quais são as regras do problema;
2. qual é a lógica algorítmica necessária;
3. quais conceitos de data e hora do .NET são realmente necessários;
4. como esses tipos representam os conceitos do meu domínio;
5. quais casos extremos podem quebrar minha lógica;
6. somente depois disso, como transformar a solução em código.

Quero que a IA me ajude principalmente como **professor, revisor e consultor técnico**, e não como substituta do meu raciocínio.

Quando eu apresentar uma ideia ou implementação própria:

- priorize avaliar minha lógica;
- aponte inconsistências e casos problemáticos;
- explique por que uma abordagem é melhor ou pior;
- sugira testes ou casos extremos;
- evite reescrever toda a solução sem necessidade;
- não forneça código completo imediatamente, a menos que eu peça explicitamente.

Quando eu estiver aprendendo uma funcionalidade do .NET, explique primeiro o conceito e sua relação com meu problema atual. Evite apresentar uma lista extensa de recursos que não são necessários para este projeto.

10. Escopo de estudo de data e hora

Quero estudar somente os conceitos necessários para resolver este problema.

`TimeSpan`

Preciso entender principalmente:

- representação de duração;
- criação de durações em minutos;
- `FromMinutes` e operações equivalentes;
- soma e subtração;
- comparação;
- cálculo da duração de um intervalo;
- `TotalMinutes` e diferença para propriedades como `Minutes`.

Aplicação no projeto:

duração desejada
duração de intervalo livre
comparação entre as duas

`DateTimeOffset`

Preciso entender:

- o que representa um instante;
- o significado do offset;
- diferença entre horário local, offset e UTC;
- criação e comparação;
- operações com `TimeSpan`;
- `AddMinutes`, `AddHours`, `Add`;
- `Subtract`;

- conversão para UTC;
- comportamento das comparações quando os valores possuem offsets diferentes;
- implicações de usar UTC nas entradas e saídas da API.

Quero compreender especialmente que `DateTimeOffset` representa um instante associado a um offset, mas não representa sozinho um identificador de fuso horário.

`DateOnly`

Preciso entender:

- representação de uma data sem horário;
- comparação;
- `AddDays`;
- cálculo da diferença entre datas;
- iteração entre datas;
- uso para representar o período de pesquisa por dias.

Aplicação no projeto:

```
02/09/2026
03/09/2026
04/09/2026
...
```

`TimeOnly`

Preciso entender:

- representação de um horário do dia;
- comparação;
- operações com horários;
- combinação conceitual entre data e horário;
- limitações de `TimeOnly` para representar intervalos que atravessam a meia-noite.

Aplicação no projeto:

```
08:00
18:00
23:30
06:00
```


TimeZoneInfo

Preciso estudar apenas o necessário para:

- compreender a diferença entre offset e fuso horário;
- converter corretamente entre UTC e horário local;
- entender como um fuso pode possuir regras próprias;
- determinar se a utilização de `TimeZoneInfo` é realmente necessária no escopo atual.

Não quero aprofundar inicialmente em todos os recursos relacionados a fusos horários.

11. Conhecimentos conceituais mais importantes

Além das classes do .NET, quero compreender operações sobre intervalos temporais.

Preciso saber raciocinar sobre:

- duração de um intervalo;
- comparação entre intervalos;
- sobreposição;
- interseção;
- ordenação;
- intervalos consecutivos;
- identificação de lacunas;
- intervalo que atravessa a meia-noite;
- diferença entre um dia como unidade de consulta e um intervalo contínuo usado para cálculo.

O objetivo não é decorar métodos, mas entender suficientemente essas operações para conseguir implementar e testar a lógica por conta própria.

12. Como quero que você conduza meu aprendizado

Antes de sugerir código, verifique se eu consigo explicar a lógica com minhas próprias palavras.

Quando possível:

1. apresente o conceito;
2. relacione-o diretamente ao meu problema;
3. mostre pequenos exemplos;
4. proponha um caso para eu raciocinar;
5. deixe a implementação principal para mim;
6. revise meu resultado.

Se eu estiver tomando uma decisão de arquitetura ou domínio, questione a decisão quando houver um problema real, mas não introduza complexidade apenas porque existe uma solução mais sofisticada.

Meu objetivo é aprender **o suficiente para resolver corretamente o problema atual**, mantendo consciência de como a solução poderia evoluir futuramente.

A prioridade é equilibrar:

```
autonomia  
+  
entendimento  
+  
uso consciente de IA  
+  
tempo de desenvolvimento  
+  
qualidade da solução
```

Não quero aprender tudo sobre data e hora do C#. Quero aprender exatamente o que é necessário para compreender e construir esta funcionalidade de forma tecnicamente correta.